



汇编语言与微机接口技术

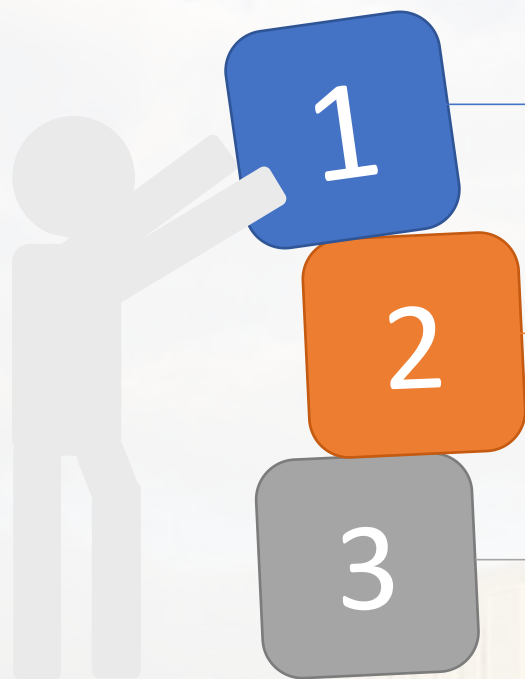
第4章 汇编语言程序设计

2025年2月10日



信息与软件工程学院
School of Information and Software Engineering

- 01. 汇编语言基本要求
- 02. 常用伪指令介绍
- 03. 程序的段结构定义
- 04. 汇编语言源程序的基本框架
- 05. 汇编语言程序应用实例



不同的汇编程序有不同的汇编语言编程规定。

目前支持Intel 8086/8088系列微机常用的汇编程序有ASM、**MASM**、**TASM**、**OPTASM**等。

本章主要介绍汇编语言程序设计中的基本书写格式与语法规则。

微软，波兰得公司等，**MASM**:
Micro Assembly Language

4.1

汇编语言语句种类 及其格式

4.1 汇编语言语句种类及其格式



4.1 汇编语言语句种类及其格式

指令语句

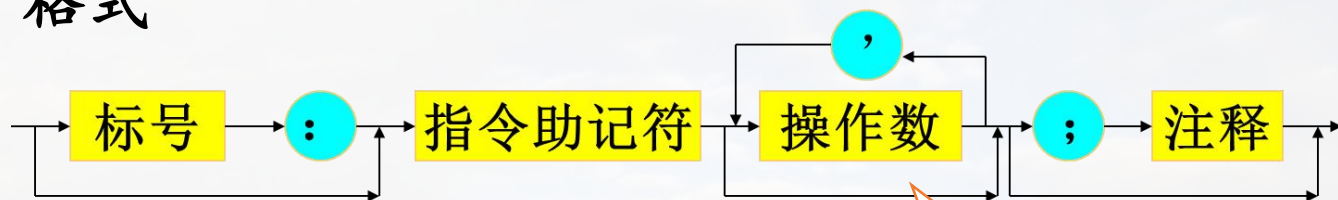


功能

每一条指令语句在汇编时都要产生一条可供CPU执行的**机器目标代码**，它又叫可执行语句。



格式



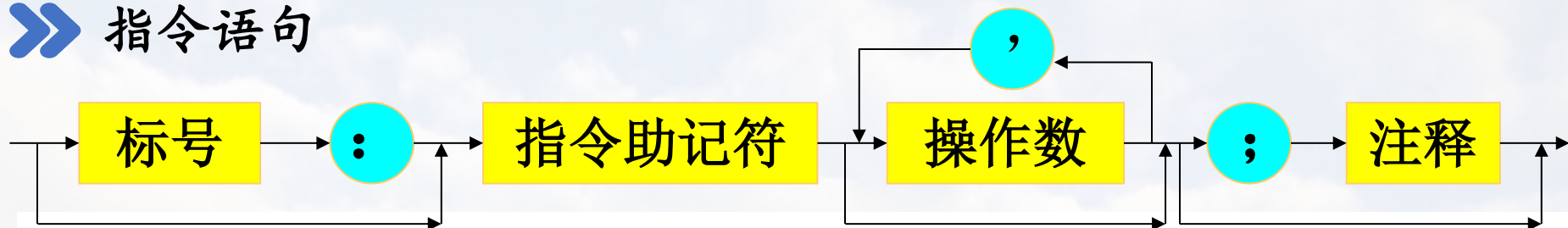
说明

一条指令语句最多可以包含4个字段

操作数：0、1、2

4.1 汇编语言语句种类及其格式

指令语句



- ◆ **指令助记符**和**操作数**两个字段就是前面介绍的指令
- ◆ **标号**是可选字段，后面必须跟“:”。主要用于**控制程序执行顺序**。
- ◆ **注释字段**为可选项，该字段以分号“;”开始。
 - ✓ 不会产生机器目标代码，它不影响程序的功能。
 - ✓ 注释可以加在指令的后面，也可以是整个语句行。

例: LABEL1: ADD AX, BX; **功能为** $AX \leq (AX) + (BX)$; 后面的程序段将完成一次对存储器的访问

.....

4.1 汇编语言语句种类及其格式

伪指令语句

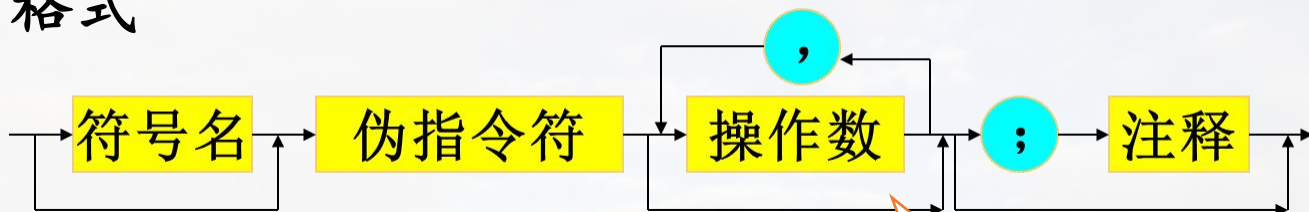


说明

- 伪指令语句又叫命令语句, 指示性语句。
- 伪指令本身不产生对应的机器目标代码。它指示汇编程序对其后面的指令语句和伪指令语句如何处理。



格式



操作数: 1~若干

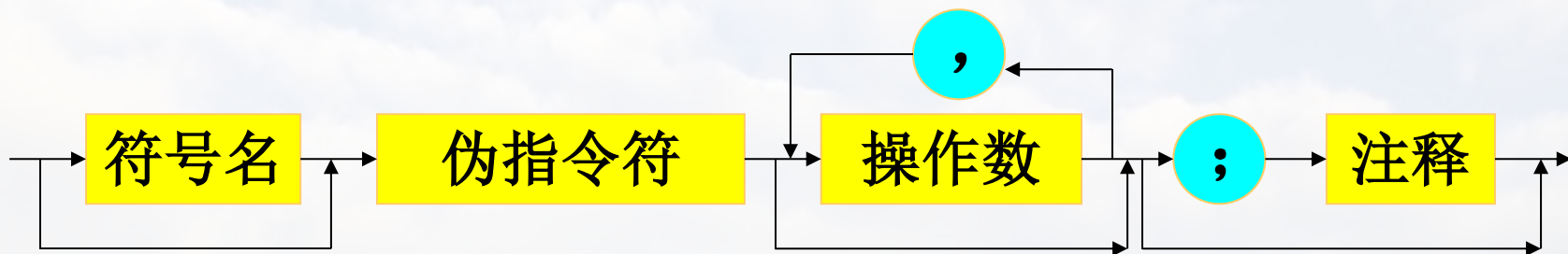


其它

一条伪指令语句最多可以包含四个字段

4.1 汇编语言语句种类及其格式

伪指令语句



- 符号名、伪指令符和操作数这三个字段就构成了伪指令，是本章后面介绍的主要内容。
- 注释字段与指令语句相同。

4.1 汇编语言语句种类及其格式

» 标识符

指令语句中的标号和伪指令语句中符号名统称为**标识符**。

标识符由若干个字符构成，构成规则：

-  1. 字符的个数为1~31个；

MASM可以超过31个
-  2. 第一个字符必须是字母、问号、@或下划线“_”这4种字符之一；

MASM可以首字母为0~9的数字
-  3. 从第二个字符开始，可以是字母、数字、@、“_”或问号“?”；
-  4. 不能使用系统专用的保留字。

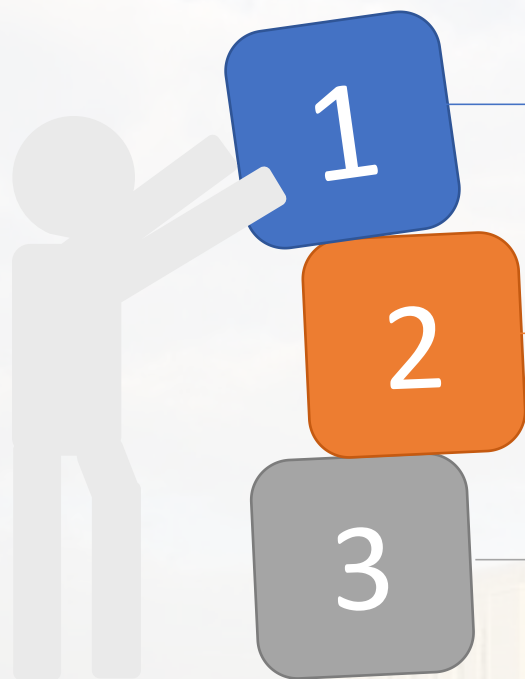
4.1 汇编语言语句种类及其格式

» 标识符



4.2

汇编语言数据



数据：指令和伪指令语句中**操作数**。

常用的数据形式有：**常数、变量和标号**。

一个数据由**数值和属性**（比如是字节数据还是字数据）两部分构成。

» 常数

常数：经过汇编后其值已完全确定，并且在程序运行过程中，其值不会发生变化。

常数的表示形式：

-  二进制数：以字母B结尾，如01001001B
-  八进制数：以字母O或Q结尾，如631Q、254O
-  十进制数：以字母D结尾，或者没有结尾字母。
如2007D、2007。
-  十六进制数：以字母H结尾，如3FEH。
 - 如果常数的第一个数符为字母，为了与标识符区别，必须在其前面冠以数字“0”。

4.2 汇编语言数据

» 常数

■ 实数。格式：

± 整数部分 • 小数部分 E ± 指数部分

例 2.134 E +10

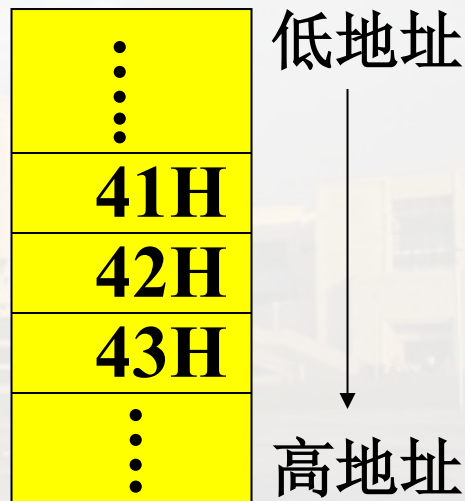
尾数

组成原理IEEE754格式：
32（短实型）、64、80位

汇编程序在汇编源程序时，可以把实数转换为4字节、8字节或10字节的二进制数形式存放。

■ 字符串常数：用引号（单引号或双引号）括起来的一个或多个字符，其值为这些字符的ASCII码值。

例如，字符串`ABC`的值为41H、42H、43H。在内存中的存储如图所示。



常数在程序的使用

使用

01

作指令语句的源操作数（立即寻址）

```
MOV    AX,    0B2F0H
ADD    AH,    64H
```

在指令语句的直接寻址方式，寄存器相对寻址方式或基址-变址-相对寻址方式中作位移量。

02

```
MOV    BX,    [32H]
MOV    0ABH[BX], CX
ADC    DX,    1234H[BP][DI]
```

03

在数据定义伪指令中使用

```
DB    10H
DW    3210H
```


» 变量

1

变量：用来表示存放数据的**存储单元**，这些数据在程序运行期间可以被改变。

2

程序中以变量名的形式来访问变量。**变量名就是存放数据的存储单元地址。**

4.2 汇编语言数据

» 变量的定义与预置

定义变量：给变量在内存中分配一定的存储单元（或空间）。也就是给这个存储单元赋予一个符号名，即变量名，同时还要将这些存储单元预置初值。

定义变量使用数据定义伪指令 DB、DW、DD、DQ和DT等

DB: Define Byte
DW: Define Word
DD: Define Double Word
.....

变量的定义与预置格式



变量名	DB	表达式1, 表达式2.....;	定义字节变量
	DW	表达式1, 表达式2.....;	定义字变量
	DD	表达式1, 表达式2.....;	定义4字节变量
	DQ	表达式1, 表达式2.....;	定义8字节变量
	DT	表达式1, 表达式2.....;	定义10字节变量



说明

其中表达式1、表达式2是给存储单元赋的初值。



举例：

```
VAR_DATA SEGMENT
    DATA1 DB 12H
    DATA2 DB 20H, 30H
    DATA3 DW 5678H
VAR_DATA ENDS
```

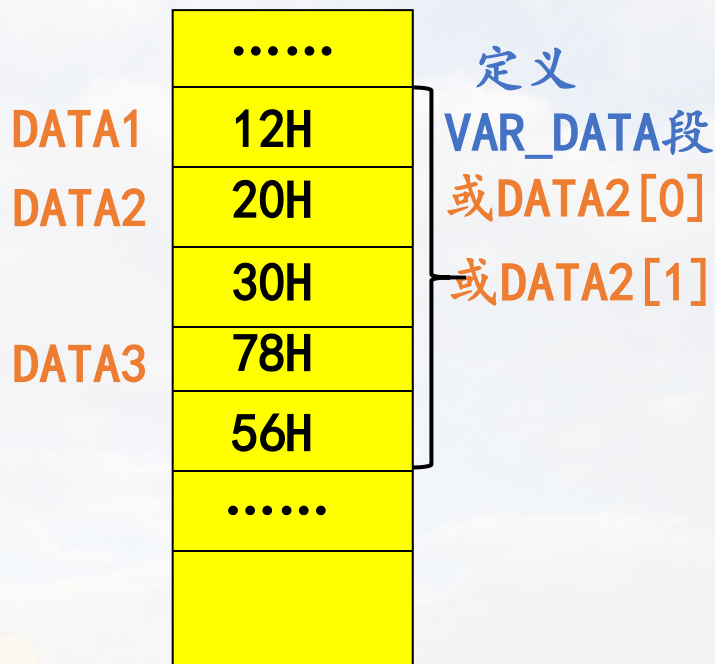
4.2 汇编语言数据

» 举例

```

例如: VAR_DATA SEGMENT
      DATA1    DB    12H
      DATA2    DB    20H, 30H
      DATA3    DW    5678H
VAR_DATA ENDS
    
```

存储器中地址空间分配



» 变量的定义与预置

当变量被定义后，就具有了以下三个属性：



4.2 汇编语言数据

» 变量的定义与预置

段属性 (SEG)

- 它表示变量存放在哪一个逻辑段中。
- 例如上面例子中的变量DATA1、DATA2和DATA3三个变量都存放在VAR_DATA逻辑段中。

偏移量属性 (OFFSET)

- 表示变量所在位置与段起始点之间的字节数。
- 上例中，变量DATA1的偏移量为0，DATA2为1，DATA3为3。

段属性和偏移量属性就构造了变量的逻辑地址

类型属性 (DB、DW等)

表示变量占用存储单元的字节数。

- DB伪指令定义的变量为1字节
- DW定义的变量为字 (2字节)
- DD定义的为双字 (4字节)
- DQ定义的为4字 (8字节)
- DT定义的为5字 (10字节)

4.2 汇编语言数据

在变量定义语句中给变量赋初值的形式



数值表达式：赋初值给变量

例如：DATA1 DB 32, 30H

DATA1[0] 单元内容为32（即20H），DATA1[1]单元内容为30H.



? 表达式赋初值给变量：表示可以预置任意内容

例如：DA_BYTE DB ?, ?, ?

表示让汇编程序分配三个字节存储单元。这些存储单元的内容的值为任意。



字符串表达式：赋初值给变量

对于DB伪指令：

- 字符串用引号括起来，长度不超过255个字符。
- 字符串的每个字符分配一个字节单元，从左到右将各字符的ASCII码以地址递增的顺序依次存放。

字符串存放在M中，是按其ASCII码存放的

4.2 汇编语言数据

» 举例

最多255个字符

例如: **STRING1 DB** 'ABCDEF'

STRING1

41H	'A'
42H	'B'
43H	'C'
44H	'D'
45H	'E'
46H	'F'

例如: **STRING2 DW** 'AB', 'CD', 'EF'

STRING2

42H	'B'
41H	'A'
44H	'D'
43H	'C'
46H	'F'
45H	'E'

对于**DW**伪指令: 可以给最多两个字符组成的字符串分配两个字节存储单元。

注意: 两个字符的存放顺序是前一个字符放在高地址, 后一字符放低地址单元。

4.2 汇编语言数据

» 举例

● 对于DD伪指令

注意：不匹配，比较特殊

- 只能给最多两个字符组成的字符串分配4个字节单元。
- 两个字符存放在较低地址的两个字节单元中，存放顺序与DW伪指令相同。
- 而较高地址的两个字节单元存放0。

给2个字符分配4个字节；怎样给1个字符分配？

例如：STRING3 DD 'AB' , 'CD'

STRING3

42H	‘B’
41H	‘A’
0	
0	
44H	‘D’
43H	‘C’
0	
0	

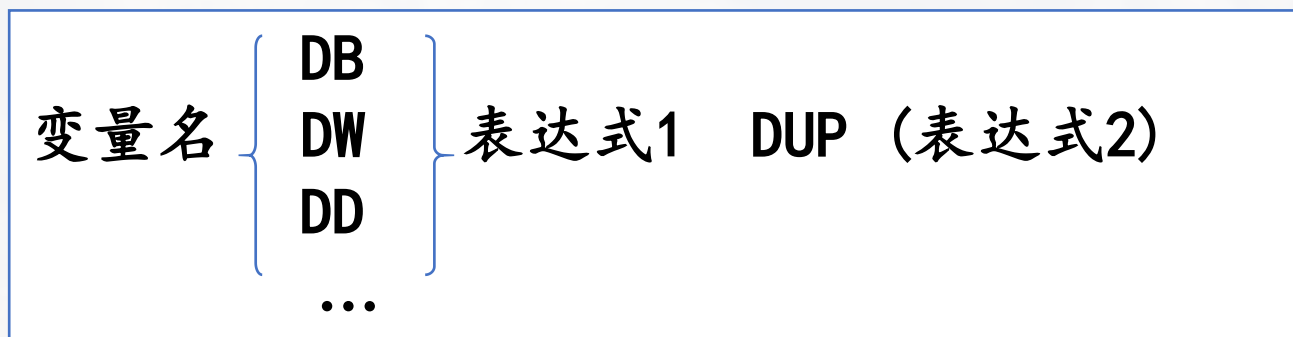
注意：DW和DD伪指令给变量赋值不能用两个以上字符构成的字符串赋初值，否则将出错；而DB则可以。

4.2 汇编语言数据

» 在变量定义语句中给变量赋初值的形式

 **DUP**表达式: **DUP**称为重复数据操作符。

使用DUP表达式的一般格式:



其中: 表达式1是重复的次数, 表达式2是重复的内容。

例如: DATA_A DB 10H DUP(?)

分配16个字节单元

DATA_B DB 20H DUP('AB')

分配20H*2=40H个字节, 其内容为重复字符串'AB'。

在变量定义语句中给变量赋初值的形式



DUP表达式

DUP还可以嵌套使用，即表达式2又可以是一个带DUP的表达式。

例如：DATA_C DB 10H DUP (4 DUP (2), 7)

重复10H个数字序列“2，2，2，2，7”，共占用10H*5=50H个字节。

» 变量的使用



» 在指令语句中引用

例如：DA1 DB OFEH

DA2 DW 52ACH

.....

MOV AL, DA1

;将OFEH传送到AL中

MOV BX, DA2

;将52ACH传送到BX中

在指令语句中直接引用变量名就是对其存储单元的内容进行存取

4.2 汇编语言数据

在指令语句中引用

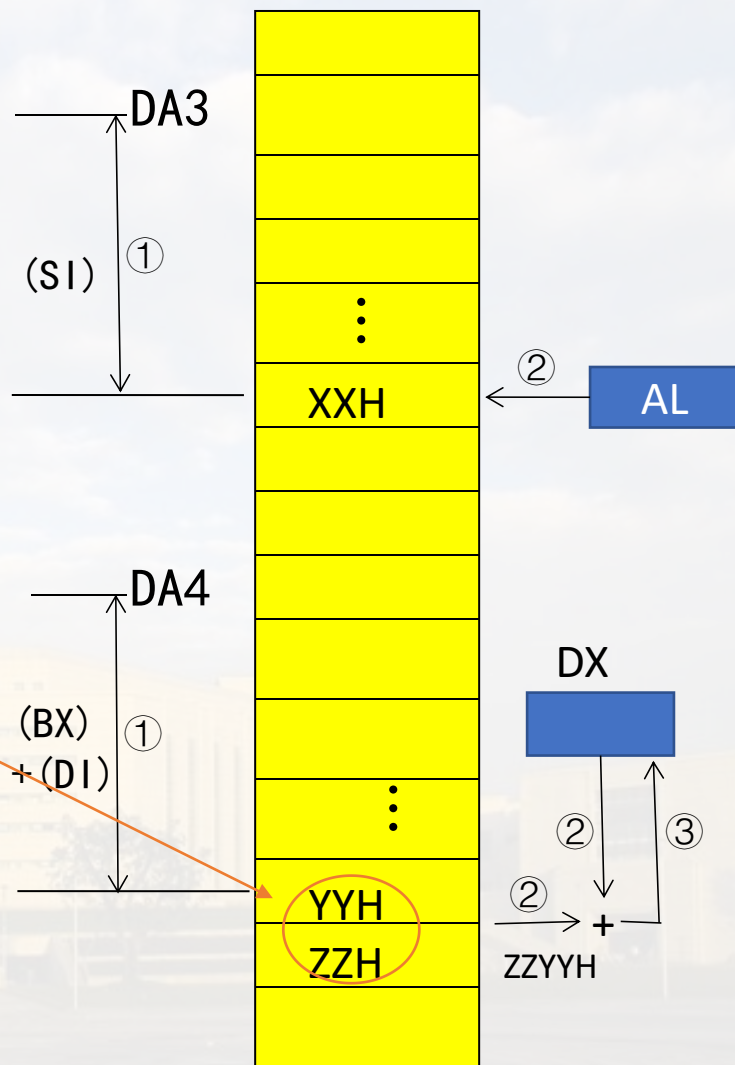
当变量出现在寄存器相对寻址或基址-变址-相对寻址的操作数中时，表示取用**该变量**的**偏移量**。

例如：

```
DA3 DB 10H DUP (?)
DA4 DW 10H DUP (1)
MOV DA3[SI], AL
ADD DX, DA4[BX][DI]
```

将从DA4开始再位移(BX)+(DI)的**字**存储单元的内容与DX的内容相加，结果送回DX中

将AL的内容送入从DA3开始再位移(SI)的存储单元中



4.2 汇编语言数据

在伪指令语句中引用

```
NUM    DB    75H
ARRAY  DW    20H DUP (0)
ADR1   DW   NUM
ADR2   DD   NUM
ADR3   DW   ARRAY[2]
```

它表示取变量的偏移地址

取变量段基址和偏移地址。前两个字节存偏移地址，后两个字节存段基址

后面三条伪指令的操作数中包含了前面定义的两个变量

设上述语句所在段的段基址为0915H，NUM的偏移地址为0004H，则存储单元的分配情况如图所示。

4.2 汇编语言数据

在伪指令语句中引用

```
NUM    DB    75H
ARRAY  DW    20H DUP (0)
ADR1   DW   NUM
ADR2   DD   NUM
ADR3   DW   ARRAY[2]
```

它表示取变量的偏移地址

取变量段基址和偏移地址。
前两个字节存偏移地址，后
两个字节存段基址

后面三条伪指令的操作数中包含了
前面定义的两个变量

设上述语句所在段的段基址为0915H，NUM的偏移地址为0004H，则存储单元的分配情况如图所示。

注意：变量不能出现在DB语句的右边，因为变量的偏移地址为16位，与DB定义的变量长度不匹配。

NUM	75H
ARRAY+0	00
+1	00
:	:
+3F	00
ADR1	04
	00
ADR2	04
	00
	15H
	09H
ADR3	07
	00
	:

总结：变量的用法

1. 在指令语句中的用法：

- 1) 直接引用，对该存储单元内容进行存取；
- 2) 在寄存器相对寻址，或基址-变址-相对寻址中，当作获取M中操作数的相对位移量使用。

2. 在伪指令语句中的用法：

格式：

- 1) 变量X DW 变量1: 取变量1的偏移地址(共16位) → 变量存储X单元(2B中)
- 2) 变量X DD 变量2: 取变量2的段基值+偏移地址(共32位) → 变量存储X单元(4B中)

» 标号



标号加在一条指令的前面，它就是该指令在内存的存放地址的符号表示，也就是指令地址的别名。



标号主要用在程序中需要改变程序的**执行顺序**时，用来**标记转移的目的地**。

```
例如：    MOV    CX, 100
          LAB:  MOV    AX, BX
          .....
          LOOP   LAB
          JNE    NEXT    ;不为零转移
          .....
          NEXT:  .....
```

» 标号的属性（与变量的三属性比较）



段属性（SEG）

它表示该标号所代表的地址在哪个逻辑段中，即段基值。



偏移量属性（OFFSET）

它表示该标号所代表的地址在段内与段起点间的字节数，即地址的偏移量。



距离属性（也叫类型属性）

它表示该标号可以被段内还是段间的指令调用。

NEAR（近）：NEAR标号只能作段内转移，即只能是与该标号所指指令同在一个逻辑段的其它指令才能使用它。

FAR（远）：FAR标号可以被非本段的转移和调用指令使用。

标号的属性

标号的距离属性
可以有两种方法
来指定

隐含方式

当标号加在指令语句前面时，它隐含为NEAR属性。

例 SUB1: MOV AX, 30H

SUB1的距离属性为NEAR，它只能被本段的转移指令和调用指令访问。

用LABEL伪指令给标号指定距离属性

格式：标号名 LABEL 类型

类型为NEAR或FAR。该语句要与指令语句连用。

例如： SUB1_FAR LABEL FAR
SUB1: MOV AX,30H

.....

- ◆ SUB1_FAR与SUB1两个标号具有相同的逻辑地址。被转移指令或调用指令使用时是指同一个入口地址。
- ◆ SUB1只能被本段调用，SUB1_FAR可以被其它段的指令调用。

» 标号的属性

LABEL伪指令还可用来定义变量的属性，即改变一个变量的属性，如把字变量的高低字节作为字节变量来处理。

例如：`DATA_BYTE LABEL BYTE`
`DATA_WORD DW 20H DUP ( ? )`

- `DATA_BYTE`与`DATA_WORD`具有相同的段基址和偏移量。
- `DATA_BYTE`可以被用来存取一个字节数据，而`DATA_WORD`则不能。

4.3

符号定义语句

» 符号定义语句

符号定义语句将常数或表达式等形式用某个指定的符号来表示。
在8086/8088汇编语言中有两种符号定义语句。

4.3 符号定义语句

» 等值语句



格式

符号名 EQU 表达式



功能

用符号名来表示EQU右边的表达式或常数。后面的程序中一旦出现该符号名，汇编程序将把它替换成该表达式。

表达式可以是任何形式，常见的有以下几种情况。

4.3 符号定义语句

» 等值语句

常数或数值表达式

例如: `COUNT EQU 5`

`NUM EQU COUNT+5`

地址表达式

例如: `ADR1 EQU DS: [BP+14]`

ADR1被定义为在DS数据段中以BP作基址寻址的一个存储单元。

变量、寄存器名或指令助记符

例如: `CREG EQU CX`; 在后面的程序使用CREG就是使用CX

`CBD EQU DAA`; DAA为十进制调整指令。

注意: 在同一源程序中, 同一符号不能用EQU定义多次。

例:

<code>CBD</code>	<code>EQU</code>	<code>DAA</code>	} 错误用法
<code>CBD</code>	<code>EQU</code>	<code>ADD</code>	

4.3 符号定义语句

» 等号语句



格式

符号名=表达式

(等号语句与等值语句具有相同的作用。但等号语句可以对一个符号进行多次定义。)



举例

例如: `CONT=5`
`NUM=14H`
`NUM=NUM+10H`



错误示范

`CBD=DAA`

.....

`CBD=ADD`

等号语句不能为助记符定义别名

注意：等值语句与等号语句都不会为符号分配存储单元。所定义的符号没有段、偏移量和类型等属性。

4.4

表达式与运算符

4.4 表达式与运算符

» 表达式与运算符

表达式是指令或伪指令语句中操作数的常见形式。它由常数、变量、标号等通过操作运算符连接而成。

注意：任何表达式的值在程序被汇编的过程中进行计算确定，而不是到程序运行时才计算。

操作运算符分类



4.4 表达式与运算符

» 算术运算符

$+$ 、 $-$ 、 $\times$ 、 $/$ 、MOD、SHL、SHR、 $[]$

1. 运算符“ $+$ ”和“ $-$ ”也可作单目运算符，表示数的正负。
2. 使用“ $+$ ”、“ $-$ ”、“ $\times$ ”、和“ $/$ ”运算符时，参加运算的数和运算结果都是**整数**。
3. “ $/$ ”运算为取**商的整数部分**，而“MOD”运算取除法运算的**余数**。

例如：

NUM=15 X 8	; NUM=120
NUM=NUM/7	; NUM=17
NUM=NUM MOD 3	; NUM=2
NUM=NUM+5	; NUM=7
NUM= -NUM-3	; NUM=-10
NUM= -NUM-NUM	; NUM=20

4.4 表达式与运算符

算术运算符

4. “SHR” 和 “SHL” 为逻辑移位运算符

“SHR” 为右移，左边移出来的空位用0补入。

“SHL” 为左移，右边移出来的空位用0补入。

注意：移位运算符与移位指令区别。

- 移位运算符的操作对象是某一具体的数（常数），在汇编时完成移位操作。
- 而移位指令是对一个寄存器或存储单元内容在程序运行时执行移位操作。

例如

NUM=11011011B

.....

不能改成：SHL NUM,1

MOV AX , NUM **SHL** 1

MOV BX , NUM **SHR** 2

ADD DX , NUM **SHR** 6

左边的指令序列等效下面三条指令。

MOV AX , **10110110B**

MOV BX , **00110110B**

ADD DX , **3**

移位运算将对象按照与AX等长的16bit数处理

4.4 表达式与运算符

» 算术运算符

5. 下标运算符 “[]” 具有相加的作用

一般使用格式：表达式1 [表达式2]

作用：将表达式1与表达式2的值相加后形成一个存储器操作数的地址。

下面两个语句是等效的。

```
MOV    AX, DA_WORD[20H]
MOV    AX, [DA_WORD+20H]
```

下面是几个错误的语句。

```
MOV    AX, ARRAY+BX+SI
MOV    AX, ARRAY+BX[SI]
MOV    AX, ARRAY+DA_WORD
```

可以用寄存器来存放下标变量

例：下面几个语句是等价的

```
MOV    AX ,   ARRAY[BX][SI]; 基址变址相对寻址
MOV    AX ,   ARRAY[BX+SI]
MOV    AX ,   [ARRAY+BX][SI]
MOV    AX ,   [ARRAY+SI][BX]
MOV    AX ,   [ARRAY+BX+SI]
```


4.4 表达式与运算符

逻辑运算符

逻辑运算符有NOT、AND、OR和XOR等四个，它们执行的都是按位逻辑运算。

例如

MOV **AX** , NOT 0F0H

=>MOV AX , 0FF0FH

MOV **AL** , NOT 0F0H

=>MOV AL , 0FH

MOV BL , 55H AND 0F0H

=>MOV BL , 50H

MOV BH , 55H OR 0F0H

=>MOV BH , 0F5H

MOV CL , 55H XOR 0F0H

=>MOV CL , 0A5H

注意：结果不一样！

4.4 表达式与运算符

关系运算符

关系运算符包括：EQ（等于）、NE（不等于）、LT（小于）、LE（小于等于）、GT（大于）、GE（大于等于）

说明

01

关系运算符用来比较两个表达式的大小。比较的两个表达式**必须同为常数或同一逻辑段中的变量**。

02

若是常量的比较，则按无符号数进行比较；若是变量的比较，则比较它们的偏移量的大小。

OFFH或OFFFH (8/16位)

03

关系运算的结果只能是“真”（全1）或“假”（全0）

4.4 表达式与运算符

关系运算符例

例1: `MOV AX, 0FH EQ 1111B` $\Rightarrow$ `MOV AX, 0FFFFH`
`MOV BX, 0FH NE 1111B` $\Rightarrow$ `MOV BX, 0`

NUM的16位偏移地址取出来比较

例2: `VAR DW NUM LT 0ABH`

该语句在汇编时，根据符号常量NUM的大小来决定VAR存储单元的值，当 $\text{NUM} < 0\text{ABH}$ 时，则变量VAR的内容为 0FFFFH ，否则VAR的内容为0。

» 数值返回运算符

该类运算符有5个，它们将变量或标号的某些特征值或存储单元地址的一部分提取出来。

4.4 表达式与运算符

数值返回运算符

1. SEG运算符：取变量或标号所在段的段基址。

例如：

```
DATA SEGMENT
    K1 DW 1, 2
    K2 DW 3, 4

DATA ENDS
.....
MOV AX, SEG K1
MOV BX, SEG K2
```

设DATA逻辑段的段基址为1FFEh，则两条传送指令将被汇编为：

```
MOV AX, 1FFEh
MOV BX, 1FFEh
```

4.4 表达式与运算符

数值返回运算符

2. OFFSET运算符：该运算符的作用是取变量或标号在段内的偏移地址。

例如：

```
DATA SEGMENT
```

```
    VAR1    DB    20H DUP (0)
```

```
    VAR2    DW    5A49H
```

```
    ADDR    DW    VAR2
```

```
DATA    ENDS
```

```
.....
```

```
MOV     BX,  VAR2;
```

```
MOV     SI,  OFFSET  VAR2
```

```
MOV     DI,  ADDR
```

```
MOV     BP,  OFFSET  ADDR
```

4.4 表达式与运算符

数值返回运算符

2. OFFSET运算符：该运算符的作用是取变量或标号在段内的偏移地址。

例如：

DATA SEGMENT

VAR1 DB 20H DUP (0)

VAR2 DW 5A49H

ADDR DW VAR2 ;将VAR2的偏移量20H存入ADDR中

DATA ENDS

.....

MOV BX, VAR2; (BX)=5A49H

MOV SI, OFFSET VAR2 ;(SI)=0020H

MOV DI, ADDR ;(DI)=0020H

MOV BP, OFFSET ADDR ;(BP)=0022H

4.4 表达式与运算符

数值返回运算符

3. TYPE运算符：取**变量或标号**的类型属性，并用数字形式表示。对变量来说就是取它的字节长度。

变量 {
 BYTE 1
 WORD 2
 DWORD 4
 QWORD 8
 TWORD 10

标号 {
 NEAR -1
 FAR -2

例如：

```
V1 DB 'ABCDE'  
V2 DW 1234H, 5678H  
V3 DD V2  
.....  
MOV AL, TYPE V1  
MOV CL, TYPE V2  
MOV CH, TYPE V3
```

经汇编后的等效指令序列如下：

```
MOV AL, 1  
MOV CL, 2  
MOV CH, 4
```


4.4 表达式与运算符

数值返回运算符

4. LENGTH运算符：该运算符用于取变量的长度。

1. 不能取标号；
2. 标号未给出长度的单位：如DB、DW等说明。

- 如果变量是用重复数据操作符DUP说明的，则LENGTH运算取最外层DUP给定的值。
- 如果没有用DUP说明，则LENGTH运算返回值总是1。

例如

```
K1  DB  10H DUP (0)
K2  DB  10H, 20H, 30H, 40H
K3  DW  20H DUP (0, 1, 2 DUP (0) )
K4  DB  'ABCDEFGH'
.....
```

无DUP说明，取1

取最外层DUP值

无DUP说明，取1

```
MOV  AL,  LENGTH  K1; (AL)=10H
MOV  BL,  LENGTH  K2 ; (BL)=1
MOV  CX,  LENGTH  K3 ; (CX)=20H
MOV  DX,  LENGTH  K4 ; (DX)=1
```

4.4 表达式与运算符

数值返回运算符

5. SIZE运算符：该运算符只能用于变量，SIZE取值等于LENGTH和TYPE两个运算符返回值的乘积。

1. 不能取标号；
2. 与TYPE有关，统一用B表示长度单位。

例如，对于上面例子，加上以下指令：

```
MOV AL, SIZE K1 ; (AL) =10H
MOV BL, SIZE K2 ; (BL) =1
MOV CL, SIZE K3 ; (CL) =20H*2=40H
MOV DL, SIZE K4 ; (DL) =1
```

» 属性修改运算符

这一类运算符用来对变量、标号或存储器操作数的类型属性进行修改或指定。



4.4 表达式与运算符

属性修改运算符

1. PTR运算符：将地址表达式所指定的**标号**、**变量**或用其它形式表示的**存储器地址**的类型属性修改为“类型”所指的**值**。

格式：**类型** **PTR** **地址表达式**

标号修改

类型可以是BYTE、WORD、DWORD、NEAR和FAR。这种修改是**临时**的，只在含有该运算符的语句内有效。

变量、存储器数修改

例如：

```
DA_BYTE  DB  20H  DUP (0)
DA_WORD   DW  30H  DUP (0)
.....
MOV  AX, WORD PTR  DA_BYTE[10]
ADD  BYTE PTR  DA_WORD[20], BL
INC  BYTE PTR  [BX]
SUB  WORD PTR  [SI], 100
JMP  FAR  PTR  SUB1;指明SUB1不是本段中的地址
```


4.4 表达式与运算符

属性修改运算符

2. HIGH/LOW运算符：这两个运算符用来将表达式的值分离出高字节和低字节。

格式：

HIGH	表达式
LOW	表达式

只能分离16位数

- 如果表达式为一个常量，则将其分离成高8位和低8位
- 如果表达式是一个地址（段基值或偏移量）时，则分离出它的高字节和低字节。

4.4 表达式与运算符

属性修改运算符

2. HIGH/LOW运算符：这两个运算符用来将表达式的值分离出高字节和低字节。

例如：

```
DATA    SEGMENT
    CONST    EQU    0ABCDH
    DA1      DB      10H DUP    (0)
    DA2      DW      20H DUP    (0)
DATA     ENDS
.....
MOV     AH, HIGH    CONST
MOV     AL, LOW     CONST
MOV     BH, HIGH    (OFFSET DA1)
MOV     BL, LOW     (OFFSET DA2)
MOV     CH, HIGH    (SEG DA1)
MOV     CL, LOW     (SEG DA2)
```

设DATA段的段基值是0926H，则指令序列汇编后的等效指令为：

```
MOV     AH , 0ABH
MOV     AL , 0CDH
MOV     BH , 00H
MOV     BL , 10H
MOV     CH , 09H
MOV     CL , 26H
```

汇编后，马上变为常数；非执行指令时再处理

4.4 表达式与运算符

属性修改运算符

2. HIGH/LOW运算符：这两个运算符用来将表达式的值分离出高字节和低字节。

注意： HIGH/LOW运算符不能用来分离一个变量、寄存器或存储器单元的内容，只能放在一个常量前面。

下面语句中的用法是错误的。

```
DA1 DW 1234H
```

```
.....
```

```
MOV AH, HIGH DA1; 变量
```

```
MOV BH, LOW AX; 寄存器
```

```
MOV CH, HIGH [SI]; 存储器
```

4.4 表达式与运算符

» 运算符的优先级

优先级别	运算符
(最高) 1	LENGTH, SIZE , 圆括号
2	PTR, OFFSET, SEG, TYPE, THIS
3	HIGH, LOW
4	X , /, MOD, SHR, SHL
5	+, -
6	EQ, NE, LT, LE, GT, GE
7	NOT
8	AND
(最低) 9	OR, XOR

» 运算符的优先级

汇编程序在计算表达式时的处理规则：

先执行优先级别高的运算，再算较低级别运算；

01

相同优先级别的操作，按照在表达式中的顺序，从左到右进行；

02

可以用圆括号改变运算的顺序。

03

例如：

K1=10 OR 5 AND 1

； 结果为K1=11

K2= (10 OR 5) AND 1

； 结果为K2=1

4.5

程序的段结构

4.5 程序的段结构



8086/8088将内存按逻辑段进行管理，不同的逻辑段可以用来存放不同目的的内容。

在程序中使用四个段寄存器CS, DS, ES和SS来访问它们。

在源程序中，使用伪指令来定义和使用这些逻辑段。

80386以后，扩展段MS、GS等

4.5 程序的段结构

» 段定义伪指令

伪指令**SEGMENT**和**ENDS**用于定义一个逻辑段。使用时必须**配对**，分别表示定义的开始与结束。

一般格式:

段中数据起始位置

段连接

```
段名  SEGMENT  [定位类型] [组合类型] ['类别名']  
.....  
... .. 本段语句序列  
... ..  
段名  ENDS
```


» 段定义伪指令语句各部分的作用

段名

01

段名由用户自己任意选定，但要符合标识符定义规则

02

最好选用与该逻辑段用途相关的名称。如第一个数据段为DATA1，第二个数据为DATA2等。

03

一对**SEGMENT**和**ENDS**前的**段名**必须一致。

4.5 程序的段结构

段定义伪指令语句各部分的作用

段基地址不一定与起始数据边界重合

定位类型用于决定**段的起始数据边界**，即第一个可存放数据的位置（不一定是段基地址）。它可以有4种取值。

PAGE:表示该段从一个页面的边界开始存放数据

由于一个页面为256个字节，并且页面编号从0开始，因此，PAGE定位类型的段起始地址的最后8位二进制数一定为0，即以00H结尾的地址。

01

PARA:表示该段从一个小节的边界开始存放数据

02

如果用户未选定位类型，则缺省为PARA；其段起始地址的最后4位二进制数一定为0，即以0H结尾的地址。

03

WORD:表示该段从一个偶数字节地址开始存放数据，即段起始数据单元地址的最后一位二进制数一定是0b。

定位
类型

04

BYTE:表示该段起始数据单元地址可以是任一地址值。

4.5 程序的段结构

» 段定义伪指令语句各部分的作用

组合类型说明符用来指定段与段之间的连接关系和定位。它有六种取值选择。

默认使用

01

若**未指定组合类型**，表示本段与其它段无连接关系。在装入内存时，本段有自己的物理段，因此有自己的段基址

02

PUBLIC: 在满足定位类型的前提下，将与该段同名的段邻接在一起，形成一个新的逻辑段，共用一个段基址。段内的所有偏移量调整为相对于新逻辑段的段基址。

03

COMMON: 产生一个覆盖段。在多个模块连接时，把该段与其它也用**COMMON**说明的同名段置成相同的段基址，这样可达到共享同一存储区。共享存储区的长度由同名段中最大的段确定。

组合
类型

4.5 程序的段结构

» 段定义伪指令语句各部分的作用

组合类型说明符用来指定段与段之间的连接关系和定位。它有六种取值选择。

组合类型

04

STACK: 把所有同名段连接成一个连续段, 且系统**自动**对SS段寄存器初始化为该连续段的段基址, 并初始化堆栈指针SP。

用户程序中应至少有一个段用STACK说明, 否则需要用用户程序自己初始化SS和SP。

05

AT表达式: 表示本段可定位在表达式所指示的小节边界上。表达式的值也就是段基值。

06

MEMORY: 表示本段在存储器中应定位在所有其它段之后的最高地址上。如果有多个用MEMORY说明的段, 则只处理第一个用MEMORY说明的段, 其余的被视为COMMON。

» 段定义伪指令语句各部分的作用

类别名

01

类别名为某一个段或几个相同类型段设定类型名称。

02

系统在进行连接处理时，把类别名相同的段存放在相邻的存储区，但段的划分与使用仍按原来的设定。

03

类别名必须用单引号引起来。所用字符串可任意选定，但它不能使用程序中的标号、变量名或其它定义的符号。

在定义一个段时，段名是必不可缺的项，而定位类型、组合类型和类别名三个参数是可选项。各个参数之间用空格分隔。各参数之间的顺序不能改变。

4.5 程序的段结构

» 段定义伪指令语句各部分的作用

分段结构的源程序框架：

```
STACK1    SEGMENT    PARA    STACK    'STACK0'
           .....
STACK1    ENDS
DATA1     SEGMENT    PARA    'DATA'
           .....
DATA1     ENDS
STACK2    SEGMENT    PARA    'STACK0'
           .....
STACK2    ENDS
CODE      SEGMENT    PARA    MEMORY
          ASSUME     CS:CODE, DS:DATA1, SS:STACK1
MAIN:     .....
          .....
CODE      ENDS
DATA2     SEGMENT    BYTE    'DATA'
           .....
DATA2     ENDS
          END    MAIN
```

4.5 程序的段结构

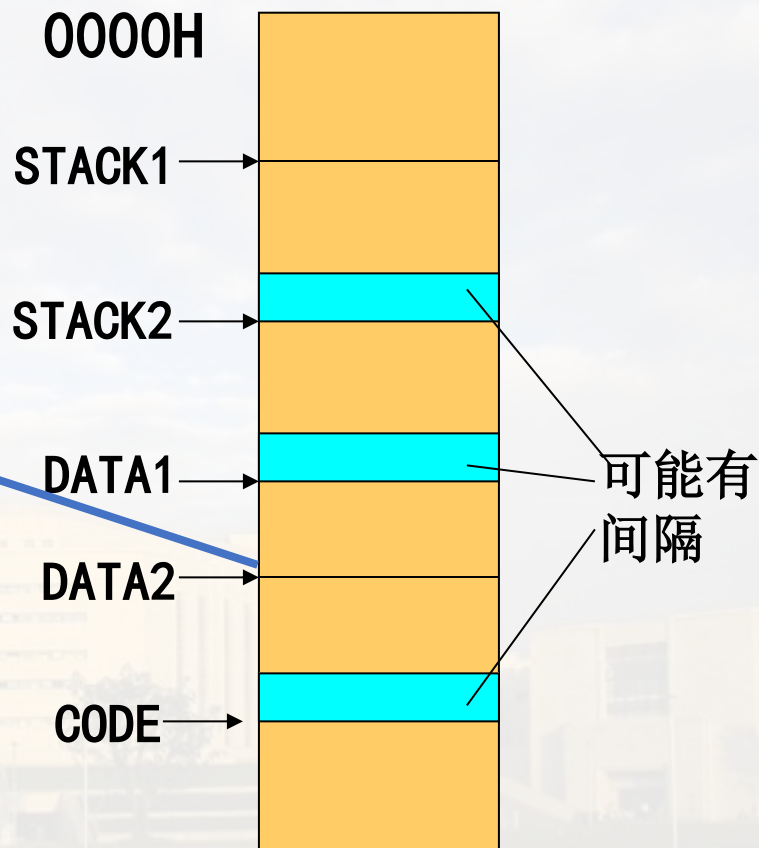
段定义伪指令语句各部分的作用

上述源程序经LINK程序进行连接处理后，程序被装入内存的情况如图所示。

- 如果在段定义中选用了PARA说明，则在一个段的结尾与另一个段的开始之间可能存在一些空白，图中以蓝色框表示。
- CODE段的组合类型为MEMORY，因此被装入在其它段之后。

如果此地址不是小节开始，则DATA2的段基址不在这个位置，而在前面的小节开始处。

在进行程序设计时，如果程序不大，一般只需要定义三个段就可以了，即**CS**、**DS**、**SS**三个段。



4.5 程序的段结构

» 段设定伪指令ASSUME



说明

- **ASSUME**的作用是告诉汇编程序,在处理源程序时,定义的段与哪个段寄存器关联。
- **ASSUME**并不设置各个段寄存器的具体内容, 段寄存器的值是在程序运行时设定的。



格式

ASSUME 段寄存器名: 段名, 段寄存器名: 段名,

其中段寄存器名为CS, DS, ES和SS四个之一, 段名是用**SEGMENT/ENDS**伪指令定义的段名。

ASSUME 段寄存器:段名  汇编时, 只是关联, 未赋值, 赋值是在程序运行时

4.5 程序的段结构

» 段设定伪指令ASSUME例

```
DATA1 SEGMENT
    VAR1  DB 12H
DATA1 ENDS
DATA2 SEGMENT
    VAR2  DB 34H
DATA2 ENDS
CODE  SEGMENT
    VAR3  DB 56H
    ASSUME CS:CODE,DS:DATA1,ES:DATA2
START: .....
    ....
    INC VAR1
    INC VAR2
    INC VAR3
    .....
CODE  ENDS
END START
```

该指令汇编时，VAR1使用的是DS

该指令被汇编时，VAR2使用的是ES，即指令编码中有段前缀ES

该指令汇编时，VAR3使用的是CS，即指令编码中有段前缀CS

4.5 程序的段结构

» 段设定伪指令ASSUME例

说明

01

在一个代码段中可以有几条ASSUME伪指令，对于前面的设置，可以用ASSUME改变原来的设置。

02

一条ASSUME语句不一定设置全部段寄存器，可以选择其中一个或几个段寄存器。

03

可以使用关键字NOTHING将前面的设置删除。

例如：

ASSUME ES:NOTHING ;删除前面对ES与某个定义段的关联

ASSUME NOTHING ;删除全部4个段寄存器的设置

» 段寄存器的装入

要让一个段寄存器真正地指向某个逻辑段，就需要将段基值装入到该段寄存器。

段寄存器的装入需要用程序的方法来实现。四个段寄存器的装入方法略有不同。

4.5 程序的段结构

段寄存器的装入

1. **DS和ES**的装入：在程序中，使用数据传送语句来实现对DS和ES的装入。

例如：

```
DATA1 SEGMENT
    DBYTE1 DB 12H
DATA1 ENDS
DATA2 SEGMENT
    DBYTE2 DB 14H DUP(?)
DATA2 ENDS
CODE SEGMENT
    ASSUME CS:CODE,DS:DATA1
START: MOV AX,DATA1
        MOV DS,AX
        MOV AX,DATA2
        MOV ES,AX
        MOV AL,DBYTE1
        MOV DBYTE2[2],AL
        .....
CODE ENDS
END MAIN
```

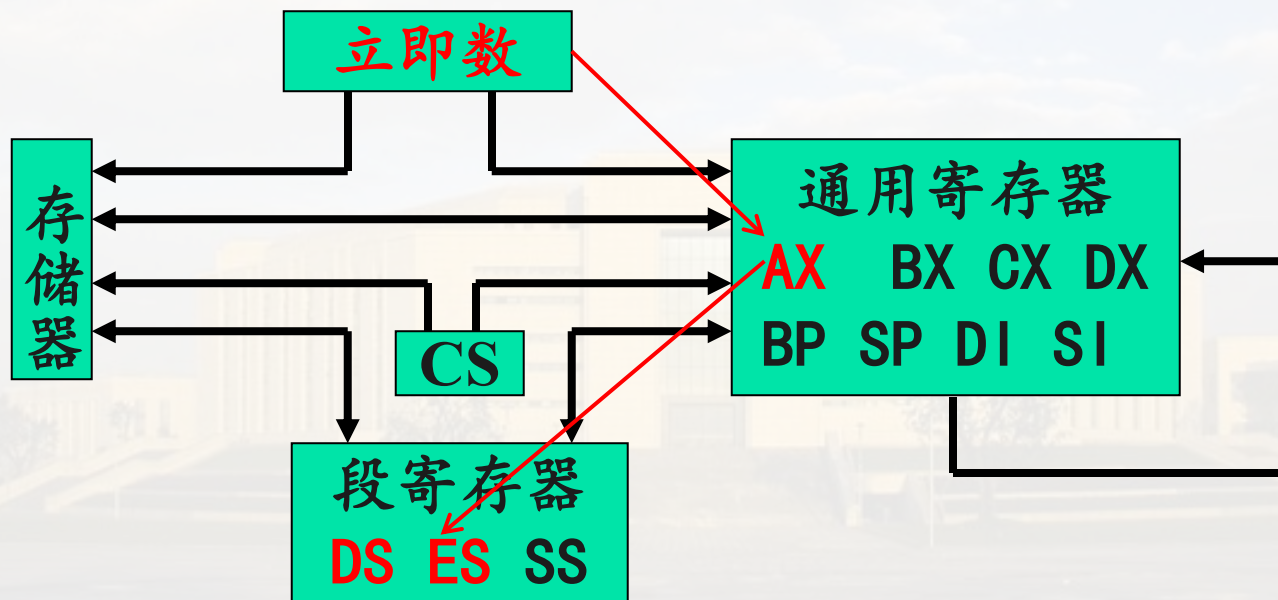
该指令在汇编时出错，因为在
ASSUME指令中未指定ES与DATA2关联

4.5 程序的段结构

段寄存器的装入

1. DS和ES的装入：在程序中，使用数据传送语句来实现对DS和ES的装入。

在汇编语言程序中，引用段名就是以立即数形式获取该逻辑段的段基址，而在MOV指令中，立即数不能直传给段寄存器，所以给段寄存器的赋值必须经过一个通用R中转，再送入DS/ES。



4.5 程序的段结构

» 段寄存器的装入

1. DS和ES的装入：在程序中，使用数据传送语句来实现对DS和ES的装入。

为了改正上述程序中的错误，可以在变量DBYTE2前加一个段前缀说明即可。即：

```
MOV ES:DBYTE2[2], AL
```

但上述方法只是在一条指令中有效，最好的办法是在ASUMME语句中，将ES与DATA2关联。

4.5 程序的段结构

段寄存器的装入

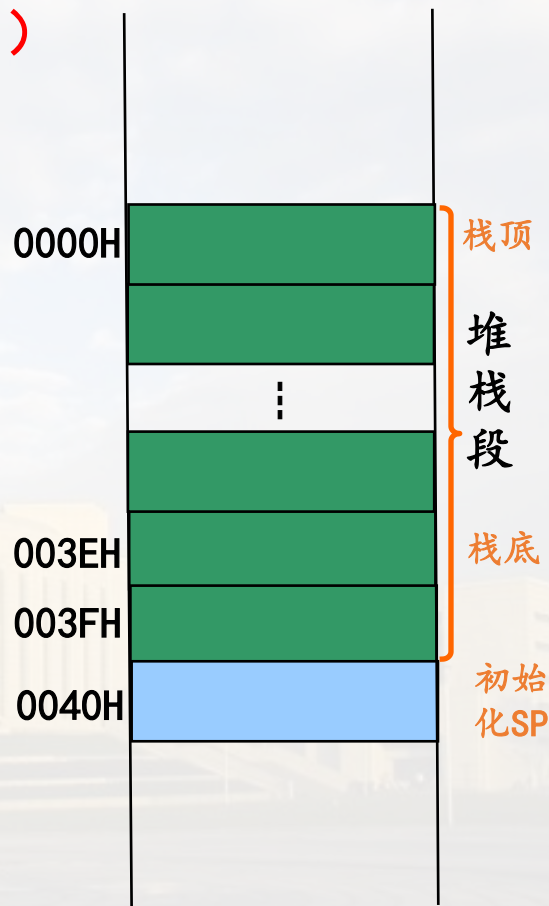
2. SS的装入：SS的装入有两种方法

参看本节PPT72，段定义伪指令对组合类型的要求

(1) 在段定义伪指令的组合类型项中，使用STACK参数，并在段寻址伪指令ASSUME语句中，把该段与SS段寄存器关联。（常用）

例如：

```
STACK1 SEGMENT PARA STACK
    DB 40H DUP (?)
STACK1 ENDS
.....
CODE SEGMENT
ASSUME CS:CODE, SS:STACK1
.....
```



SS将被自动装入STACK1段的段基值，堆栈指针SP也将指向堆栈底部+2的存储单元。上例中 (SP) = 40H。

4.5 程序的段结构

段寄存器的装入

2. SS的装入：SS的装入有两种方法

(2) 如果在段定义伪指令的组合类型中，未使用STACK参数，或者是在程序上要调换到另一个堆栈，使用类似于DS和ES的装入方法。 (不要求)

例如：

```
DATA_STACK SEGMENT
    DB 40H DUP (?)
    TOP LABEL WORD
DATA_STACK ENDS

.....
CODE SEGMENT
    .....
    MOV AX, DATA_STACK
    MOV SS, AX
    MOV SP, OFFSET TOP
    .....
```

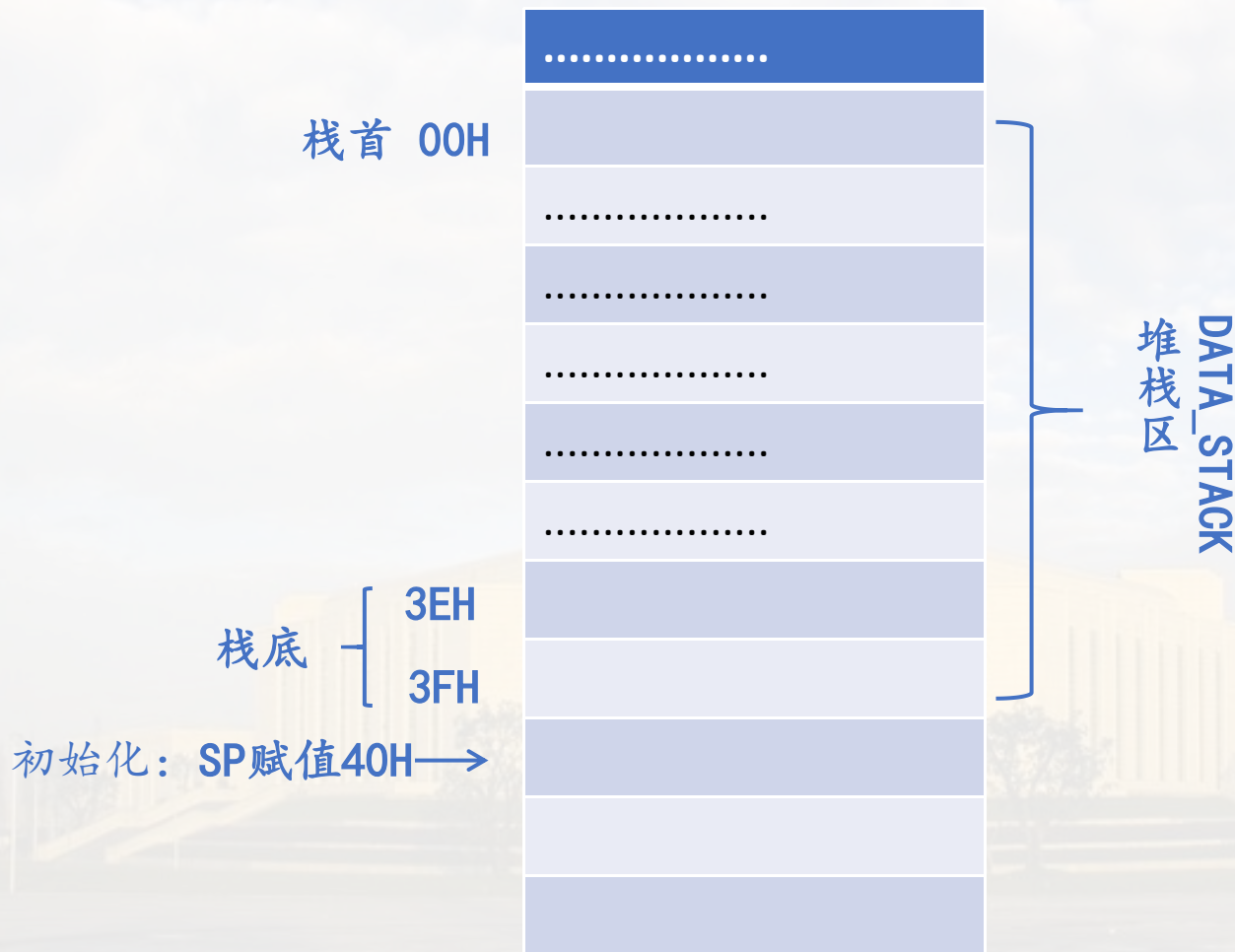
TOP变量的偏移量为40H，即栈底+2

SP是按字操作的

4.5 程序的段结构

» 段寄存器的装入

2. SS的装入：初始化示意图



4.5 程序的段结构

段寄存器的装入

3. CS的装入：CPU在执行指令之前根据CS和IP的内容来从内存中提取指令，即必须在程序执行第1条语句之前装入CS和IP的值。因此，CS和IP的初始值就不能用可执行语句来装入。装入CS和IP一般有以下两种情况。

(1) 由系统软件按照结束伪指令指定的地址装入初始的CS和IP（常用）

任何一个源程序都必须以END伪指令来结束。

格式：END 起始地址

段定义结束用：ENDS

起始地址：可以是一个标号或表达式，它与程序中第一条指令语句前所加的标号必须一致。

4.5 程序的段结构

» 段寄存器的装入

3. CS的装入：CPU在执行指令之前根据CS和IP的内容来从内存中提取指令，即必须在程序执行第1条语句之前装入CS和IP的值。因此，CS和IP的初始值就不能用可执行语句来装入。装入CS和IP一般有以下两种情况。

(1) 由系统软件按照结束伪指令指定的地址装入初始的CS和IP（常用）

- END伪指令用来指示源程序结束和指定程序运行时的起始地址。
- 当程序被装入内存时，系统软件根据起始地址的段基值和偏移量分别被装入CS和IP中。

例如：

```
.....  
CODE SEGMENT  
    ASSUME CS:CODE,.....  
    START: .....  
    .....  
CODE ENDS  
    END START
```

注意：是在CS段中

4.5 程序的段结构

» 段寄存器的装入

3. CS的装入：CPU在执行指令之前根据CS和IP的内容来从内存中提取指令，即必须在程序执行之前装入CS和IP的值。因此，CS和IP的初始值就不能用可执行语句来装入。
装入CS和IP一般有以下两种情况。

(2) 在程序运行期间，当执行某些指令时，CPU自动修改CS和IP，使它们指向新的代码段，即程序是非顺序执行中的某些类型（不包括条件转移，循环等，它们是段内转移）。

例如：

- 执行段间过程调用CALL和段间返回指令RET；
- 执行段间无条件转移指令JMP；
- 响应中断及中断返回指令；
- 执行硬件复位操作。

段间转移

4.6

过程定义伪指 (PROC/ENDP)

4.6 过程定义伪指令 (PROC/ENDP)



在程序设计过程中，常常将具有一定功能的程序段设计成一个子程序。在MASM宏汇编程序中，用过程 (PROCEDURE) 来构造子程序。



4.6 过程定义伪指令 (PROC/ENDP)

» 过程定义伪指令格式

格式

```
过程名  PROC  [NEAR/FAR]
          .....
          RET
          .....
过程名  ENDP
```

最后执行RET，但可以写在中间位置

3.6程序控制指令：过程调用和返回PPT102

过程名是子程序的名称，它被用作过程调用指令CALL的目的操作数。它类同同一个标号的作用。具有段、偏移量和距离三个属性。而距离属性使用NEAR和FAR来指定，若没有指定，则隐含为NEAR。

NEAR: 过程只能被本段指令调用

FAR: 过程可以供其它段的指令调用

每一个过程中必须包含有返回指令RET，其作用是控制CPU从该过程中返回到调用过程的主程序。

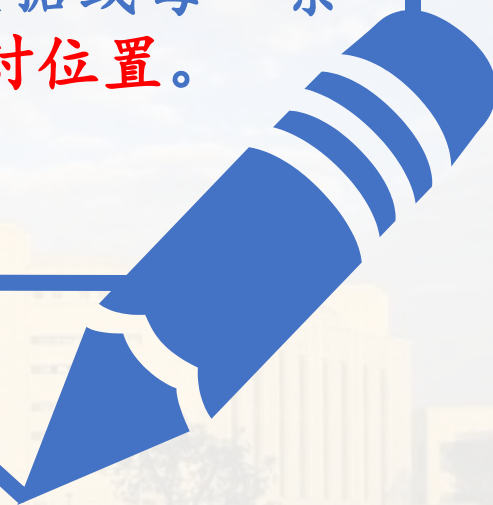
4.7

当前位置计数器\$与
定位伪指令
ORG (Origin)

4.7 当前位置计数器\$与定位伪指令ORG (Origin)



汇编程序在汇编源程序时，每遇到一个逻辑段，就要为其设置一个位置计数器，用来记录该逻辑段中定义的每一个数据或每一条指令在逻辑段中的相对位置。



4.7 当前位置计数器\$与定位伪指令ORG (Origin)



说明

01

在源程序中使用符号\$来表示位置计数器的当前值。因此，\$被称为当前计数器。

02

它位于源程序不同的位置具有不同的值。

03

位置计数器\$在使用上完全类似变量的使用：只有偏移属性。

4.7 当前位置计数器\$与定位伪指令ORG (Origin)

» 定位伪指令ORG



格式

ORG 数值表达式



作用

用来改变位置计数器的值。

将数值表达式的值赋给当前位置计数器\$。ORG语句为其后的变量或指令设置起始偏移量。



说明

表达式的值必须为正值。表达式中也可以包含有当前位置计数器的现行值\$。

4.7 当前位置计数器\$与定位伪指令ORG (Origin)

» 定位伪指令ORG例

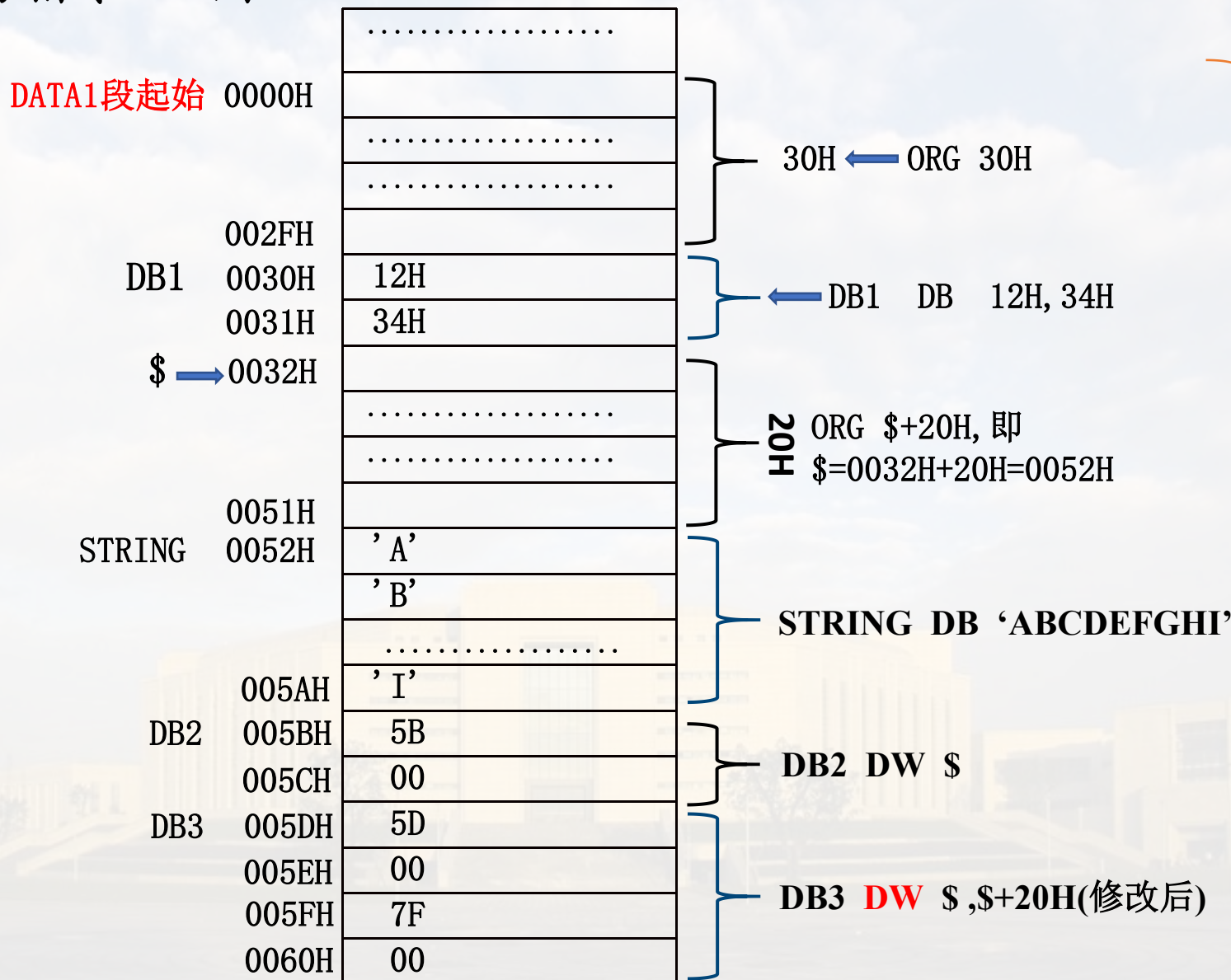
```
DATA1 SEGMENT
    ORG 30H
    DB1 DB 12H,34H    ; DB1在DATA1段内的偏移量为30H
    ORG $+20H        ; 保留20H个字节单元, 其后再存放'ABCD...'
    STRING DB 'ABCDEFGHI'
    COUNT EQU $-STRING ; 计算STRING的长度
    DB2 DW $          ; 取$的偏移量, 类似变量的用法
    DB3 DB $,$+20H    ; 此语句错误!$是16位偏移量, 不能用DB表示, 改为DW
DATA1 ENDS

CODE SEGMENT
    ASSUME CS:CODE.....
    ORG 10H
START: MOV AX,DATA1
    MOV DS,AX

    .....
CODE ENDS
END START
```


4.7 当前位置计数器\$与定位伪指令ORG (Origin)

定位伪指令ORG例



4.7 当前位置计数器\$与定位伪指令ORG (Origin)

总结：地址传送指令、变量、\$ 的用法

1. 地址传送指令：LEA、LDS/LES （参看第三章指令系统地址传送指令）

LEA REG, MEM ; 将存储器操作数的16位偏移地址取出送目标寄存器
LDS REG, MEM ; 把MEM存储单元开始的4个字节单元的内容送入
(LES REG, MEM) 目标寄存器和段寄存器 (DS/ES)

2. 在伪指令语句中变量的用法：（参看本节PPT30）

变量X DW 变量1 ; 取变量1的偏移地址(共16位) → 变量存储X单元 (2B中)

变量X DD 变量1 : 取变量1的段基值+偏移地址(共32位) → 变量存储X单元 (4B中)

3. 符号\$: 表示逻辑段中位置计数器的当前值，在使用上完全类似变量的使用，仅有偏移属性，无段属性（参看本节PPT95）

DB1 DW \$; 取\$的16位偏移地址 → DB1单元, 类似变量的用法

DB2 DB \$, \$+20H ; 此语句错误! \$是16位偏移地址, 不能用DB表示, 改为DW

DB3 DD \$, \$+20H ; 此语句错误! \$是16位偏移地址, 不能用DD表示, 改为DW

4.8

从程序返回DOS 操作系统的方法

4.8 从程序返回DOS操作系统的方法

在早期的DOS操作系统中，为了使用户的程序运行结束后，能够正确地返回到操作系统，需要在程序中加上一些必要的语句。一般有以下两种方法。



使用DOS系统功能调用
实现返回



使用程序段前缀PSP
实现返回



4.8 从程序返回DOS操作系统的方法

使用DOS系统功能调用实现返回

一、执行DOS功能调用4CH，将控制用户程序结束，并返回DOS操作系统。（常用）

在程序结束时，使用两条指令：

```
MOV AH, 4CH  
INT 21H
```

代码段的结构为：

```
CODE SEGMENT  
    ASSUME CS:CODE.....  
BEGIN:..  
    ...  
    MOV AH, 4CH  
    INT 21H  
CODE ENDS  
    END BEGIN
```

二、使用程序段前缀PSP (Program Segment Prefix) 实现返回

01 DOS系统将一个.EXE文件（可执行文件）装入内存时，在该文件的前面生成一个程序段前缀PSP。

PSP的长度为100H字节。

02

03 系统将DS和ES都指向PSP的开始。

CS指向该程序的代码段，即第一条可执行指令。

04

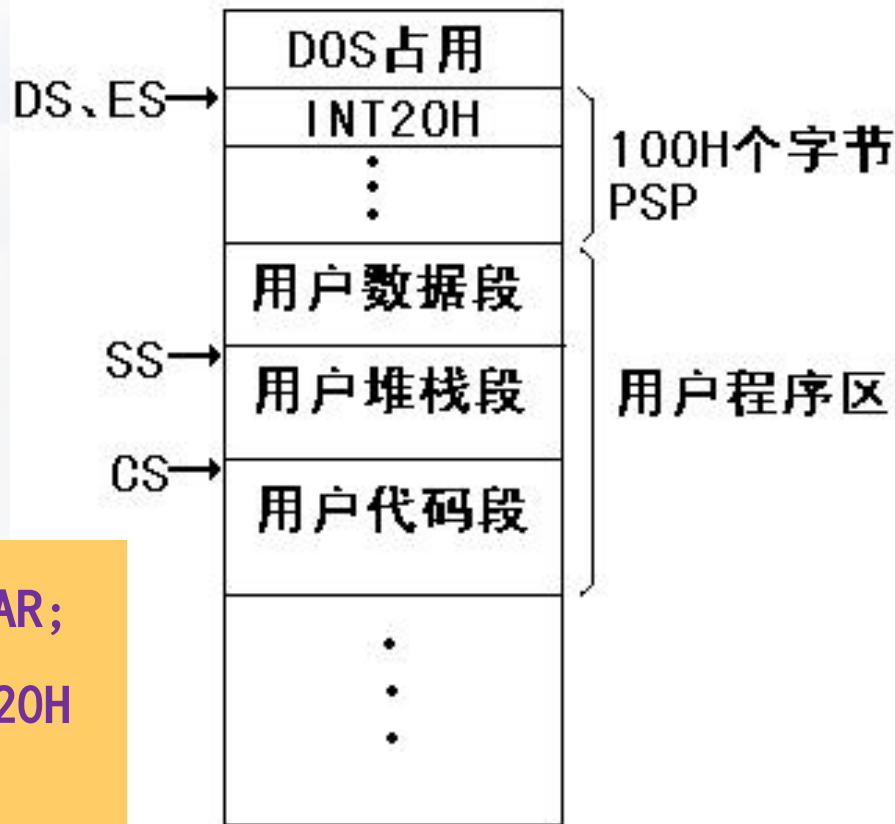
4.8 从程序返回DOS操作系统的方法

使用程序段前缀PSP (Program Segment Prefix) 实现返回

**PSP的开始是一条中断指令
INT 20H, 执行该指令将终止
用户程序, 返回DOS系统。**

为了使程序执行完后, 正确返回DOS, 需要做以下三个操作:

1. 将用户程序编制成一个过程, 类型为FAR;
2. 将PSP的起始逻辑地址压栈, 即将INT 20H指令的地址压栈;
3. 在用户程序结尾处, 使用一条RET指令。执行该指令将使保存在堆栈中的PSP的起始地址弹出到CS和IP中。



4.8 从程序返回DOS操作系统的方法

使用程序段前缀PSP (Program Segment Prefix) 实现返回

程序结构:

```
DATA    SEGMENT
...
DATA    ENDS
STACK1  SEGMENT STACK
...
STACK1  ENDS
CODE    SEGMENT
        ASSUME CS:CODE, DS:DATA, SS:STACK1
BEGIN   PROC FAR
        PUSH DS
        MOV AX, 0
        PUSH AX
        MOV AX, DATA
        MOV DS, AX
        ...
        RET
BEGIN   ENDP
CODE    ENDS
END     BEGIN
```

第一步操作BEGIN

第二步操作压栈

第三步操作返回

THANK

@刘辉
